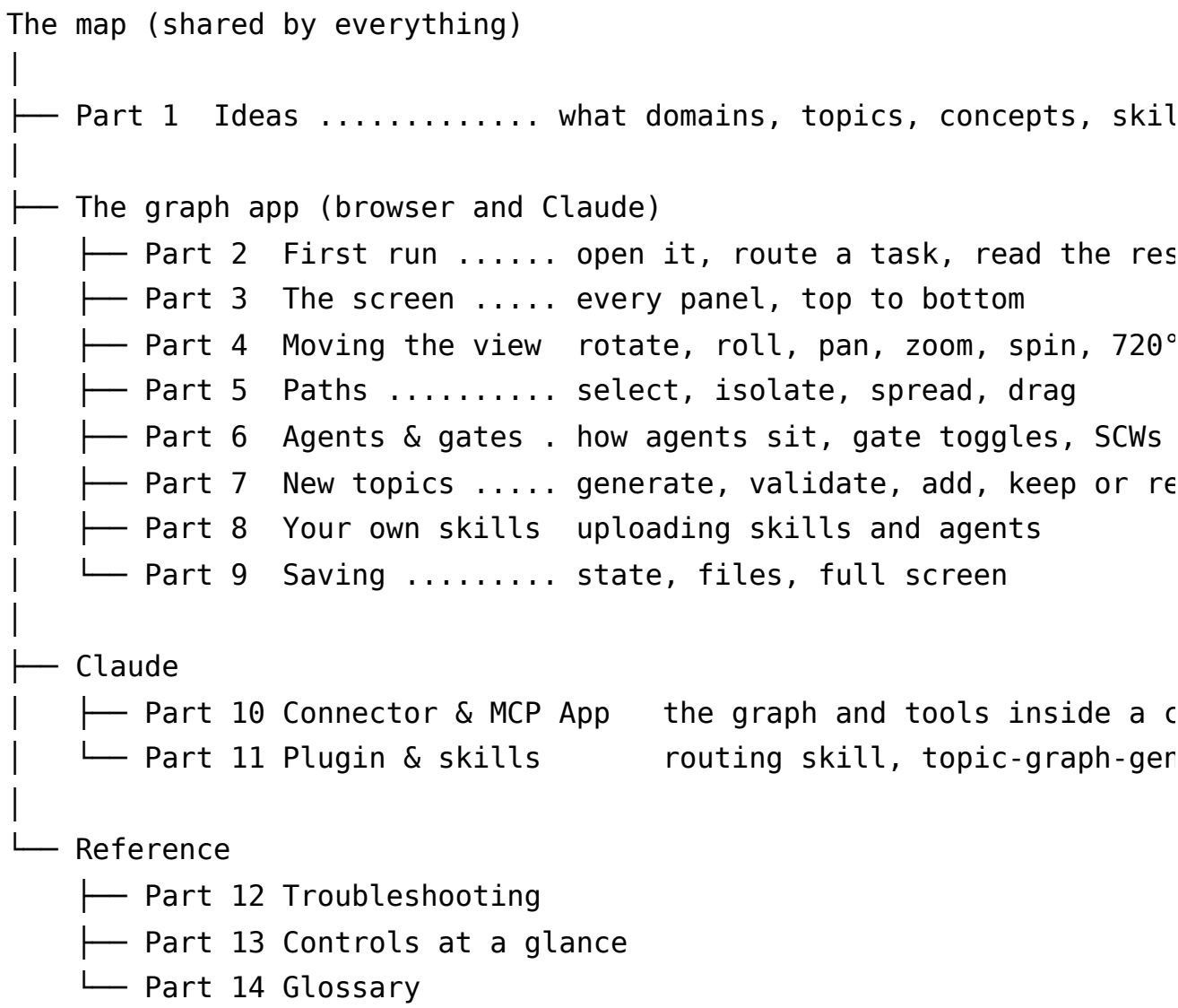
Maxey0 Context Plane — User Guide

This guide covers the three ways to use the Maxey0 Context Plane: the **graph app** in a browser, the **graph inside Claude**, and the **MCP server** behind both. It starts with the ideas you need, then walks through every part of the app, then covers Claude, the plugin and topic generation.

How this guide is organized

The product is one map shown three ways. The guide follows that shape.



Server internals and the full tool reference are in MCP_SERVER.md; how the map is built is in ARCHITECTURE.md.

Part 1 · Ideas

The six levels

Everything in the map sits on one of six levels, and every thread runs through all of them in order:

Level	What it is	Example
Domain	The broadest grouping	Finance
Topic	A field of work or study	Derivatives and Options
Concept	A distinct idea within the topic	Pricing and Greeks
Skill	A piece of work you can ask for	Option Pricer
Agent	Who should do that work	Derivatives Pricing Agent (a MaxeyN key)
MCP server	A tool that agent would use	Polygon MCP

Topics, concepts and skills each have exactly one parent. Agents and MCP servers are shared: one agent can serve skills in several concepts, and one server can be held by several agents.

The eleven domains

Domains were written from the concepts up: topics were formed from what their concepts, skills, agents and servers have in common, and domains from what their topics have in common.

Domain	Topics	Holds
Agentic Systems and Software Engineering	17	Maxey0, agent harnesses, MCP and protocols, governance, observability, state, time, reasoning, knowledge graphs, code and infrastructure
Data Science and Quantitative Methods	7	statistics and causal inference, machine learning, data engineering, visualization, optimization, decision and risk analysis, research methods
Mathematical and Physical Sciences	9	algebra, calculus, linear algebra, classical and quantum physics, optics, condensed matter, materials, chemistry and simulation
Engineering	10	electrical, chemical, mechanical, civil, aerospace and energy engineering, control, robotics and drones
Life, Health and Mind Sciences	9	cell and molecular biology, genetics, anatomy, bioengineering, biomedical computation, biomechanics, public health, psychology and cognitive science
Earth, Environment and Place	8	climate, atmosphere, water, geophysics, environmental engineering, energy and emissions policy, maps, GIS, cities, places and travel
Business, Finance and Economics	21	accounting, corporate finance, banking, markets and trading, funds, regulation, taxes, crypto, real estate, economics, startups, marketing and careers
Society, Governance and Ethics	11	politics, international relations, public policy, human rights, development, sociology, anthropology, education and the ethics of technology and data

Domain	Topics	Holds
Languages, Literatures and Communication	11	language structure, grammar, pronunciation, translation, linguistics, writing, poetry, rhetoric and literature
Area and Identity Studies	13	European, Slavic, Chinese, African, American, Latin American and Jewish studies, and the study of race, ethnicity, gender and sexuality
Humanities and Arts	12	philosophy, history, classics, religion, archaeology, art history, studio art, design, music, dance, theater, film and media

Agents: primary, checker, support

Each skill names three or four agents:

- **Primary** does the work.
- **Checker** verifies it. A checker holds every MCP server its primary holds on that skill, so it can re-run the work rather than only read it.
- **Support** helps with part of the work.

Every agent has a permanent key: **Maxey** followed by a number. **Maxey0** is the orchestrator that hands work to the others; for example, **Maxey1498** is the Quantum Circuit Agent. Keys never change between versions of the map.

Agents here are **context records**: a role, the skills it serves and the tools it would hold. Nothing in this product runs an agent or calls an MCP server. The map tells an orchestrator, or you, who should do a piece of work and with what.

MCP servers and gates

An MCP server is a tool server an agent can use. In the graph, the line from an agent to a server is a **gate**. A gate exists for one of three reasons:

- **Derived:** the server's description is similar to one of the agent's skills (cosine ≥ 0.10).
- **Inherited:** the agent is the checker on a skill whose primary holds the server.

- **Declared:** set by hand in the map's rules (one case: Maxey0 → its own session store).

Every server in the map is a real project whose existence was checked. Some can place orders, move money or change accounts; the map lists them as context only.

Routing

Routing turns a task into a path. The task's words are compared with every skill and concept using one method, TF-IDF cosine similarity: words are lower-cased, common words dropped, plurals folded, and rare words weigh more than common ones. A match needs a score of at least **0.08** and at least two shared words, so a single rare word is never enough. The best match becomes the route; the next few are listed as candidates.

Routing matches words, not meaning. If a task uses words the map doesn't, it returns no route instead of a wrong one. Rephrase with the field's own words, or generate the missing topic (Part 7).

Part 2 · First run

1. Open **maxey0-context-plane-mcp.maxey0.workers.dev/app**. The graph loads from the server in a few seconds; a notice beside the title shows progress.
2. In **Task**, type `quantum entanglement of two qubits` and press **Enter** or **Submit task**.
3. Read the green panel:
4. **Matched … best skill** names the skill and its score.
5. **Best route** reads *topic → concept → skill*: here Quantum Physics → Quantum Foundations → Entanglement Entropy Analyzer.
6. **Other candidates** are the next-best skills; click one to jump to it.
7. Look at the graph. The route is highlighted: the topic, concept and skill, the skill's agents, and those agents' MCP servers.
8. Click **Hide** on the panel to shrink it to one line. The highlight stays. Click **Show result** to bring it back.

9. Try `hello there friend` . The panel turns amber: no skill shares enough words. It offers **Generate a topic for this task** (Part 7).

Part 3 · The screen

The app is one page, top to bottom:

Header

The Human0 logo, the app name and version. **What is this graph?** opens a short explanation.

Workspace

Everything below sits in the **Workspace** panel. Its first line, **App state & full screen**, is folded by default; open it for Save, Restore, Export file, Import file, Remove state, and the browser full-screen buttons (Part 9).

Left column: Agent field control

Section	What it does
Layout & gates	Agent layout (**Skill-angle** or **Gate-field**), show or hide **skill→agent** lines and **agent→MCP gates**, **Multi-MCP only**, **Spin**, **Isolate path**, ⬡ **ISOLATE** (SCW view) and which agents get an SCW
Labels & numbering	Which levels show labels (**T** topics, **C** concepts, **S** skills, **A** agents, **M** MCP servers, **None**, **All**); agent labels as name, MaxeyN index or both; ID tools; SCW labels
Clear	Clear the **Selection** or **Search**; **Clear ALL gates** or **Restore gate defaults**
Browse & select	Search box, **Topic** and **Concept** selectors, lists of topics, concepts, agents, MCP servers or skills; **Multi-select** and **Select all listed**

Section	What it does
Overrides & export	**Reset overrides** (manual gate changes) and **Export locations (JSON)**
Upload Skill or Agent	Add your own skill or agent to the graph (Part 8)

Right column

Section	What it does
Task	Choose the router (**Flat** or **Observer lens**), type a task, **Submit task**, **Clear task**; the result panel appears below. With **Observer lens**, a readout lists for each level the nodes chosen, their weight w, their signed support ψ and each branch's contribution, plus any candidates ruled out. A domain chosen in Browse & select becomes the lens's starting point
Metrics	Live counts: topics, concepts, skills, agents, MCP servers, gate links, agents with two or more servers
Graph	The 3D graph, its toolbar and legend
Details	Everything about the selected node
Task route	The last task's top three routes, opened from **Show execution path ↓**
Topics → Concepts → Skills → Agents → MCP	Generate and manage new topics (Part 7)

Graph toolbar

Button	Does
Spread path	When a path is isolated, lay each level's nodes around its ring so every label is readable (on by default)
Drag nodes	Let a drag on a node move it (on by default)
Reset layout	Put dragged nodes back
720°	Tumble the graph through two full turns and land where it started
Full screen	Inside Claude only: ask Claude to show the app full screen
Fit view	Reset rotation, zoom and pan so the whole graph fits
Domains	Show the domain level: the top plane of the graph, one node per domain in its own colour, linked to its topics; every topic also carries its domain's colour as a ring. **D** in Labels names them; selecting a domain lights up its topics
Observer	Where the field is read from: *above the graph*, a chosen domain, topic or concept, or the routed task's lens chain
Embedding values	Select any node: Details shows its field location, its value as seen from the current observer, and its three nearest nodes on every level (the same ranking as the server's `nearest` tool; a domain counts as near as its nearest topic). Click a neighbour to jump to it. In the embedding field with an observer inside the graph, nodes the observer barely explains fade
Embedding field	Place nodes by the field. Seen from above the graph, every node sits at its location in the shared space, so nodes that mean the same thing line up across levels. Once the observer is inside the graph (a lens route, or a domain, topic or concept chosen), every plane below it is laid out from there: the chosen nodes at the centre, each other node as far out as the observer leaves it unexplained, in its field direction. Off: the ring layout

Button	Does
Show: Full graph / New topics only	Show everything, or only topics you added (Part 7)

Reading the graph

The five levels are stacked rings on one vertical axis, from the top: **Topics** (gold), **Concepts** (cyan), **Skills** (green), **Agents** (violet), **MCP servers** (pink squares). Lines run between levels: tree lines (topic → concept → skill), skill→agent lines weighted by role (primary heaviest), and gates (agent → server) shaded by strength. A ring around an agent means it holds two or more servers. Orange skills and red agents are ones you uploaded.

Part 4 · Moving the view

To	Do
Rotate in any direction	Drag empty space. The graph turns about the axis at right angles to your drag, with no limit.
Roll (tilt sideways)	**Shift** + drag
Pan	Right-drag, middle-drag or **Alt** + drag
Zoom	Scroll (0.08× to 8×)
Spin about the graph's own axis	**Spin** (Layout & gates); click again to stop
Tumble two full turns	**720°**
Start over	**Fit view**

A click on a node selects it; a click on empty space clears the selection. Hovering a node highlights its path while the pointer is on it.

Part 5 · Paths

Selecting

Click any node, pick it from **Browse & select**, or choose a **Topic** and **Concept** in the selectors. The selected node's whole path lights up: for a topic, all its concepts, skills, agents and servers; for an agent, the skills it serves and the servers it holds; for a server, the agents holding it and their skills. **Details** shows the node's description and its links.

To select several nodes, turn on **Multi-select** (or hold **Shift**/**Ctrl** and click). **Select all listed** selects everything in the current list.

Isolating

Isolate path hides everything that isn't on the selected path, so you see one thread on its own. Click it again to bring the rest back.

Spreading

With a path isolated and **Spread path** on, each level's nodes on the path are placed around that level's ring, centred under their parents, so nodes and labels no longer stack on top of each other. The spread starts on the side facing you and then turns with the graph like everything else.

Dragging

With **Drag nodes** on, drag any node to move it. A node always stays on its own level and inside that level's ring, and everything below it on the path moves with it: drag a concept and its skills, their agents and those agents' servers follow on their own rings. This works at any rotation, including upside-down. **Reset layout** puts every node back. Dragged positions are saved with the app state.

Part 6 · Agents and gates

Agent layout

- **Gate-field** (default): each agent sits at the weighted centre of the servers it holds, so agents with similar tools cluster together. Agents with no servers stay at their skill-angle position.
- **Skill-angle:** agents are spread evenly around their ring, ordered by the skills they serve.

Gate toggles

Select an agent: **Details** lists its candidate MCP gates with their scores and the reason for each (derived, inherited with the skill it came from, or declared). Tick or untick a gate to change which servers place the agent; the agent moves to its new position. **Reset overrides** undoes your changes, **Clear ALL gates** removes every gate (to see agents by skill alone) and **Restore gate defaults** brings back the map's gates. **Export locations (JSON)** saves every agent's position and gates.

Multi-MCP only dims every agent that holds fewer than two servers.

SCWs (execution boundaries)

A Structured Context Window (SCW) is the boundary where context meets execution: one agent and the servers it would use. Turn on ⬡ **ISOLATE** and select an agent to draw its SCW; Shift-click two or more agents for a multi-agent or loop SCW. **Execution SCWs** chooses whether SCWs are drawn for the selected agents, all agents, or none. **SCWn on** labels them; **Reset numbering** renumbers them and **Delete custom loops** removes the multi-agent ones you made.

Part 7 · New topics

When a task has no route, the missing piece is usually a whole topic. You can generate one with Claude, check it, add it to your graph, and keep or remove it later.

1. Build the request

Either click **Generate a topic for this task** on an amber result, or open **Topics → Concepts → Skills → Agents → MCP**, type a topic (and optionally a concept), and click **Build generation request**. The request lists the existing topics and the rules a topic must follow.

2. Generate it

Click **Ask Claude to generate it**.

- **Inside Claude**, the request goes straight into the conversation.
- **In a browser**, it is copied; paste it into Claude. Claude works best with the Maxey0 Context Plane connector or plugin installed, because it can then check its work with `validate_topic_pack`.

Claude returns one JSON object: a **topic pack** with 3–8 concepts, their skills, the agents for each skill, and only real MCP servers it could verify.

3. Validate and add

Paste the JSON into the box under the request (or import a `.json` file), click **Validate** to see problems or counts, then **Add to graph**. The new topic appears in the graph, the selectors, search and **Details**, and works with isolation, spreading, dragging and SCWs like any other topic.

4. Keep, remove, view

Each added topic is listed under **Added to this graph** with **Show**, **Remove** and **Export**. Topics are independent: add three, remove the second, and the first and third stay. **Show: New topics only** displays just the topics you added; **Full graph** brings back everything.

Added topics live in your app state (Part 9). They are not sent to the server. To propose one for the public map, export it and open an issue or pull request.

Part 8 · Your own skills and agents

Upload Skill or Agent maps your own skill or agent onto the graph:

1. Choose a type, or leave **Auto-detect**.
2. Choose files or paste a `SKILL.md`, an agent `.md`, or JSON with at least a name and a description.
3. Files you choose are staged as they load; for pasted text click **Stage pasted text**. Then click **Submit to graph for mapping** (**Clear staged** empties the list).

The app places each upload under the concept its text is most similar to. Uploaded skills are **orange**, uploaded agents **red**. Uploads are saved in your app state and can be remapped or removed from the list under the panel.

Part 9 · Saving

App state

Open **App state & full screen** (folded under Workspace):

Button	Does
Save	Saves to this browser: added topics, uploads, gate overrides, labels, filters, dragged layout, rotation, zoom and pan
Restore	Loads the saved state
Export file / **Import file**	The same, as a `.json` file you can keep or share
Remove state	Deletes the saved state from this browser

Each saved state records which version of the map it was made with. A state from a different version is refused with a message rather than loaded wrong.

Full screen

- **In a browser: Graph full screen**, **Control board full screen** or **App full screen** in the App state line.

- **Inside Claude: Full screen** in the Graph toolbar asks Claude to show the app full screen; click again to return.

Part 10 · In Claude: connector and MCP App

Connect

1. **Customize → Connectors → Add custom connector**
2. URL `https://maxey0-context-plane-mcp.maxey0.workers.dev/mcp` , no authentication.

No account is needed. To use one domain only, use `/mcp/<domain id>` instead, for example `/mcp/business-finance` or `/mcp/agentic-systems` .

Open the graph in a conversation

Ask for it:

- *"Open the Maxey0 Context Plane graph."*
- *"Open the Maxey0 Context Plane graph for the task: hedge a currency exposure."*
- *"Show the Maxey0 Context Plane graph on the Real Estate topic."*

Claude calls `open_context_plane_graph` . In hosts that support MCP Apps, such as Claude Desktop, the graph renders in the conversation with the task already routed. Elsewhere Claude gives you a link to the web app.

Ask questions without the graph

- *"Use Maxey0 Context Plane to route: reconcile a bank statement."*
- *"Which agents check skills in Quantum Physics, and which servers do they hold?"*
- *"Trace Maxey1498 back to its domain."*
- *"List the topics in the Foundations domain."*

Claude answers from the tools, fetching only what it needs. The full list of tools is in MCP_SERVER.md.

Permissions

Every tool is read-only. Whether Claude asks before each call, or before showing the app, depends on your connector permission settings in Claude.

Part 11 · Plugin and skills

The plugin **maxey0-context-plane-graph** packages the connector with two skills:

Skill	Claude uses it when
maxey0-context-plane	You ask to route a task, find who handles a skill, which MCP server fits, or to open the graph. It tells Claude which tool to call first, how to read the results (gate reasons, no-match answers) and how to report them.
topic-graph-generator	A task has no route, or you ask Claude to add or generate a topic. Claude gets the generation request, checks for overlap with existing topics, writes the pack, verifies each MCP server exists, validates the pack with `validate_topic_pack`, and returns the JSON for the app.

Install it from the plugin directory when listed, or from this repository (see the README). If you already added the connector by hand, keep one of the two so the tools don't appear twice.

Part 12 · Troubleshooting

You see	Do
The graph doesn't load; a red notice names the server	Check your connection and reload. The page needs to reach the server for `/graph.json`.
No match for a task you expect to route	Rephrase with the field's own words; or generate the missing topic (Part 7).

You see	Do
Nodes and labels pile up	Isolate the path and turn on **Spread path**, or drag nodes apart.
The graph is lost off screen	**Fit view**.
A saved state won't load	It was saved against another version of the map. Its added topics are still in the exported file: re-add them from the Topics panel.
In Claude, the graph shows as a link, not the app	That client doesn't support MCP Apps; open the link, or use Claude Desktop.
In Claude, the tools appear twice	The connector was added by hand and by the plugin. Turn one off.
Full screen missing in the toolbar	It appears only inside an MCP Apps host such as Claude; in a browser use the App state line.

Part 13 · Controls at a glance

Action	How
Route a task	Type in Task, **Enter**
Select a node	Click it, or pick it in Browse & select
Add to selection	**Shift**/**Ctrl** + click, or **Multi-select**
Isolate the selected path	**Isolate path**
Rotate any way	Drag empty space
Roll	**Shift** + drag
Pan	Right-drag, middle-drag, or **Alt** + drag

Action	How
Zoom	Scroll
Move a node on its level	Drag the node
Reset nodes / view	**Reset layout** / **Fit view**
Draw an agent's SCW	⬡ **ISOLATE**, then click the agent
Generate a topic	**Generate a topic for this task**, or the Topics panel
Save everything	App state → **Save** or **Export file**

Part 14 · Glossary

Term	Meaning
Maxey0	The orchestrator of a multi-agent system; also the one topic in the Maxey0 System domain
MaxeyN	A specialised agent's permanent key, such as Maxey1498
Context plane	The map an orchestrator reads before acting: what kinds of work exist and who handles each
SCW	Structured Context Window: a bounded workspace for one piece of work; in the graph, the boundary around an agent and its servers
SCW1	The SCW that holds this map
Topic harness	Everything about one topic in one document: its concepts, skills, agents and servers
Topic pack	The JSON format for a topic, used to build the map and to add generated topics
Gate	A link from an agent to an MCP server
Route	The best path for a task: topic → concept → skill → agents → servers

Term	Meaning
Human0	The app platform this graph belongs to

Part 15 — App 1.5 additions

- **Edge toggles.** The toolbar has one button per link level: Domain→Topic, Topic→Concept, Concept→Skill, Skill→Agent, Agent→MCP, plus Related (dashed lines between related concepts, off by default). A path that stops at skills means the Skill→Agent or Agent→MCP toggle is off.
- **Selecting a domain** shows its topics and their concepts. Skills, agents and servers appear when you select a concept or skill, or route a task.
- **Domains tab** lists all 11 domains; click one to select it.
- **Generated topics** get a suggested domain from the nearest existing topic. When you publish a pack, set `pack_domains` so the server files it under that domain.
- **Saved state** now includes router mode (flat/lens), the field view, the domain filter, every edge toggle and the browse filters.

Part 16 — App 1.6: 3D donut levels

- Each level is a donut with thickness. With **Embedding field** on, a node's place around and across the ring comes from field axes 1–2 and its height inside the donut from axis 3.
- **Drag nodes**: grab a node and move it; it follows the pointer from any viewing angle and moves in 3D within its donut. Release to place it; it stays there. **Reset layout** puts every node back.

Part 17 — App 1.7: node size and per-level framing

- Node size depends on how many nodes a level shows: a crowded level draws small nodes and a sparse one draws large nodes, so the gaps between nodes stay visible.
- With **Embedding field** on, each level fills its own donut: a node's place has no meaning by itself; what matters is its direction and distance to the other nodes on

the same level.

Part 18 — App 1.8: the lens down to MCP servers, and drift

1. Set the router to **Observer lens**, type a task, press Route.
2. The readout lists five levels: Topics, Concepts, Skills, Agents, MCP servers. For each chosen node: weight w, score ψ, **drift** (ψ minus its score seen from above: how much the path above moved it) and its rank seen from above → its rank through the lens. Each level ends with the task it received and passed on and the share it explained.
3. Turn on **Embedding field**. With **Drift arrows: on**, each chosen node has a dashed ghost where it sits seen from above and an arrow to where the observer placed it. Spin and tilt the graph; the arrows turn with it.
4. **Export drift SCW** downloads `run-N-drift-scw.json` : one SCW per chosen node, one per level, one around the whole path, with exact embedding values and the 3D home/observed places.

Part 19 — App 1.9: color scale, keyboard, drift in one frame

- **Observer color** (graph toolbar): with an observer in the graph (a lens route, or a chosen domain, topic or concept) and **Embedding field** on, nodes are colored from dark blue (the observer explains nothing of the node) to yellow (it explains the node fully). The legend in the lower-left corner shows the scale and the drift arrow colors: green = the path raised the node's fit, red = lowered it.
- **Drift arrows** now show each node's own movement: home and observed places are in one frame, and a node moves along the line between the observer and its home place, in all three axes.
- **Keyboard**: press Tab until the graph is focused (cyan outline). Right/Left = next/previous node on the level; Down = first node below; Up = first node above; Page Up/Page Down = the level above/below; Home/End = first/last on the level;

Escape = clear. Each move selects the node, and a screen reader hears its level, name, position and how many nodes are below and above.

Part 20 — App 1.12: embedding values in Details

Select any node. The **Embedding values** block lists: - **TF-IDF vector (routing, exact)**: how many of the vocabulary's terms the node uses and the largest weights (the vector has length 1). - **bge-m3 vector (routing, exact)**: the first 8 of its 1024 values (centred, length 1), **Copy all** for the whole vector, and **Nearest by bge-m3 cosine** per level. Shown when the server's dense index is available. - **Field place, 3D**: x, y, z in the shared field (SVD axes 1–3). - **Seen from above (no observer)**: where the node is drawn on its level donut, x, y, z. - **Seen from** (a lens route, or a chosen domain, topic or concept): a (how much of the node the observer explains) and √a, the observed field place and drawn place, and the **Displacement** between home and observed, x, y, z and its length relative to the donut radius. - **In this lens route**: when the route chose the node, its flat score and rank seen from above, ψ and rank through the lens, drift and weight. - **Nearest by TF-IDF cosine** per level.

Part 21 — v1.1.0: what a route will cost, before you run it

Route a task. Under the task box, **Estimated harness cost** gives the whole harness in input and output tokens and dollars, and each routed node carries its own figure on a dashed orbit:

Node	Label	What it counts
Skill	`80 tok · fixed`	The tokens of the skill's spec in the harness. It does not change with the task, model or loops.
Agent	`31k tok · $0.11 or on request`	Its turns × (CLAUDE.md + its prompt + skill specs + tool definitions of the servers it holds), plus growing history, plus its output. Primary: 4 work turns; checker: 1 review turn; support: nothing unless asked.
MCP server	`~2 calls · 2.2k tok`	Tool definitions (120 tokens per tool, assumed) plus the expected calls for this task (tools whose names share a

Node	Label	What it counts
		word with the task, at least one) at 1,000 tokens per result. "tools not listed" means the directory gives no tool names.

Controls on the same row: **Cost** (show or hide the labels), the **model** (Sonnet 5.5, Opus 5.5, Haiku 5.5, Fable 5.1, at Anthropic's list prices checked 2026-11-10) and **Loops** (maker → checker rounds; each round repeats the primary's and checker's turns).

These are estimates for planning, not a bill: no prompt caching is assumed (so real costs with caching are lower), the turn counts and result sizes are fixed assumptions, and the tool-call guess reads tool names only. Send to Claude Code writes every figure and assumption to `cost.json`.

Part 22 — v1.2.0: Coverage, Fill the gap and copy buttons

After you route a task, the **Task route** panel ends with **Coverage**:

- **Covered by this route**: task words the chosen path explains.
- **In the map but not on this route**: words a concept in the map fits well (score shown); the route ranked something else first. Try the Observer lens or say more about that part.
- **Not covered**: words nothing in the map fits; "not in the map at all" means the map has never seen the word.
- **Where it breaks**: for the uncovered phrase (the word and its neighbor, e.g. "android app"), the level where the map stops fitting. If no topic scores 0.2 or more, the proposal is a **new topic** in the best domain; if a topic fits but no concept does, a **new concept** under that topic.

Fill the gap builds a generation request for only the missing levels. What happens next depends on where you are:

Where	What Fill the gap does
Inside Claude (the graph opened through the connector)	Sends the request to Claude. Paste the pack it returns into the pack box, then **Add pack and route again**.
Dashboard with your API key	Generates the pack, adds it and routes the task again, in one step.
Dashboard without a key	Copies the request. Paste it into Claude, paste the JSON back into the pack box, then **Add pack and route again**.

Packs you add now take part in routing in this page (flat and observer lens), so the same task routes through the new topic. They live in this browser's graph; the public map changes only when a pack is added to the repository.

Every output field has a **Copy** button: the route, node details, the cost line, embedding values, status messages and the text boxes.